

Rappresentazione dell'informazione



Concetti chiave

- Sistemi di numerazione posizionale
- Basi decimale, binaria, ottale, esadecimale
- Conversioni tra basi diverse: binario/ottale/esadecimale
- Rappresentazione dei numeri in complemento a 1 e a 2
- Rappresentazione dei numeri in virgola fissa e mobile
- Codifica dell'informazione e codice ASCII

D.1 Sistemi di numerazione

Le possibili rappresentazioni dei numeri sono infinite; settantuno, LXXI, 71, 10001112 e 1078 si riferiscono allo stesso numero. Un *sistema di numerazione* è costituito da un insieme finito di simboli o cifre e di regole che permettono di rappresentare i numeri naturali. Una prima classe è quella dei *sistemi additivi* – come quelli della maggior parte dei popoli antichi: Egizi, Greci –, in cui ogni cifra, indipendentemente dalla posizione che occupa, ha un valore prefissato. Il numero si ottiene allora addizionando il valore delle cifre che lo rappresentano. Anche il sistema di numerazione romano deve essere considerato un sistema additivo, pur se alcune cifre, a seconda della loro posizione, devono essere sottratte invece che addizionate. I sistemi additivi hanno grandi limitazioni: è infatti molto scomodo, se non impossibile, rappresentare con essi numeri di una certa grandezza; inoltre non è semplice eseguire le operazioni matematiche.

La seconda classe è quella dei *sistemi di numerazione posizionale* come il nostro sistema decimale, creato in India intorno al VI secolo d.C., e che ci è arrivato tramite gli Arabi verso la fine del 1200 (è per questo motivo che le sue cifre vengono chiamate, con termine improprio, cifre *arabe*). In esso i dieci simboli/cifre:

0 1 2 3 4 5 6 7 8 9

acquistano un valore/peso diverso a seconda della posizione che occupano. Per esempio, nel numero:

$$\begin{aligned} 73093 &= 7 \cdot 10^4 + 3 \cdot 10^3 + 0 \cdot 10^2 + 9 \cdot 10^1 + 3 \cdot 10^0 = \\ &= 7 \cdot 10000 + 3 \cdot 1000 + 0 \cdot 100 + 9 \cdot 10 + 3 \cdot 1 \end{aligned}$$

la prima occorrenza di 3 vale tremila, la seconda tre. Ogni cifra deve essere moltiplicata per una potenza di 10 il cui esponente è dato dal numero delle cifre che seguono la cifra considerata. Il sistema di numerazione a *base* dieci, o *decimale*, utilizza dunque dieci simboli differenti, e si è imposto probabilmente per la corrispondenza con le dita delle mani con cui l'uomo usa contare, ma è solo uno degli infiniti sistemi possibili. I Maya in America Centrale avevano un sistema in base venti e gli Assiro-Babilonesi in base sessanta. I sistemi di elaborazione utilizzano il sistema binario perché ha il numero minore di simboli possibili ed è quindi il più semplice (anche se in passato sono state fatte sperimentazioni con basi diverse, come la base tre). Quando si parla di sistemi di numerazione in diverse basi, un numero a pedice del numero rappresentato:

$$1000111_2 \quad 107_8$$

specifica la base. Il primo dei numeri precedenti è perciò in base due, il secondo in base otto; entrambi rappresentano 71 in base dieci. La base decimale per convenzione non viene indicata.

D.2 Il sistema binario

Nei sistemi di elaborazione l'unità minima di memorizzazione può assumere solo due stati: pieno o vuoto, aperto o chiuso, vero o falso, magnetizzato o smagnetizzato, 0 o 1. Tale unità è stata chiamata *bit*, dall'inglese *Binary digiT* (cifra binaria), mentre l'insieme di otto bit viene detto *byte* (parola inglese che non ha alcun equivalente italiano) e costituisce l'unità fondamentale con cui si misurano la capacità di una memoria centrale e delle memorie periferiche. Un *byte* corrisponde allo spazio di memoria minimo necessario per contenere un qualsiasi carattere: una lettera dell'alfabeto o una cifra numerica, un segno di punteggiatura o un carattere come @ o #.

Le dimensioni delle memorie vengono comunemente indicate tramite potenze di 2 del byte, per cui si parla di *kilobyte*, *megabyte*, *gigabyte*, *terabyte* per riferirsi rispettivamente a 2^{10} , 2^{20} , 2^{30} e 2^{40} byte (Tabella DW.1). Si osservi che approssimare queste grandezze con mille, un milione, un miliardo e mille miliardi di byte, è abbastanza corretto per i kilobyte, ma non per le grandi dimensioni: per esempio

Tabella DW.1 Principali unità di misura di memoria.

kilobyte (Kb)	2^{10} byte	(1024 byte)
megabyte (Mb)	2^{20} byte	(1.048.576 byte)
gigabyte (Gb)	2^{30} byte	(1.073.741.824 byte)
terabyte (Tb)	2^{40} byte	(1.099.511.627.776 byte)

un solo gigabyte contiene oltre 70 milioni di byte in più rispetto alla sua approssimazione in base dieci. Questo fatto è sottolineato dal diverso prefisso, che per le unità di memoria è in maiuscolo (per esempio, Kb) mentre per le unità del sistema SI è in minuscolo (per esempio, kg).

Per convertire un numero binario nel suo equivalente decimale occorre moltiplicare ogni cifra per la potenza del 2 pari al numero di cifre che la seguono e poi addizionare tutti i risultati ottenuti. Per esempio:

$$\begin{aligned} 10001112 &= 1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = \\ &= 1 \cdot 64 + 0 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 71 \end{aligned}$$

Viceversa, per convertire un numero decimale nel suo equivalente binario bisogna effettuare la divisione intera del numero decimale per 2 finché non si ha come risultato zero: i resti ottenuti formano il numero binario. I resti devono essere presi in ordine inverso: l'ultimo resto ottenuto è la *cifra binaria più significativa*, il primo è la *cifra meno significativa*. Per esempio:

$$\begin{array}{ll} 71 : 2 = 35 & \text{resto } 1 \\ 35 : 2 = 17 & \text{resto } 1 \\ 17 : 2 = 8 & \text{resto } 1 \\ 8 : 2 = 4 & \text{resto } 0 \\ 4 : 2 = 2 & \text{resto } 0 \\ 2 : 2 = 1 & \text{resto } 0 \\ 1 : 2 = 0 & \text{resto } 1 \end{array}$$

La rappresentazione binaria di 71 è dunque 10001112.

D.3 I sistemi ottale ed esadecimale

I sistemi in base otto e in base sedici, detti rispettivamente *ottale* ed *esadecimale*, sono particolarmente importanti per la loro relazione con il sistema binario. Per la base sedici, essendo necessarie 16 cifre, si utilizzano per convenzione, oltre ai simboli decimali, anche le prime sei lettere dell'alfabeto (Tabella DW.2).

Per convertire in base dieci le basi otto e sedici si opera analogamente a come abbiamo fatto in precedenza:

$$\begin{aligned} 625_8 &= 6 \cdot 8^2 + 2 \cdot 8^1 + 5 \cdot 8^0 = \\ &= 6 \cdot 64 + 2 \cdot 8 + 5 \cdot 1 = 405 \\ F2B_{16} &= 15 \cdot 16^2 + 2 \cdot 16^1 + 11 \cdot 16^0 = \\ &= 15 \cdot 256 + 2 \cdot 16 + 11 \cdot 1 = 3883 \end{aligned}$$

La base sedici ci fa apprezzare come con un numero di simboli maggiore si riesca a rappresentare numeri più grandi con meno cifre. Vediamo anche la conversione inversa.

$$\begin{array}{ll} 405 : 8 = 50 & \text{resto } 5 \\ 50 : 8 = 6 & \text{resto } 2 \\ 6 : 8 = 0 & \text{resto } 6 \\ 3883 : 16 = 242 & \text{resto } 15 \end{array}$$

Tabella DW.2 Tabella di conversione tra diversi sistemi.

Decimale	Binario	Ottale	Esadecimale
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

$242 : 16 = 15$ resto 2
 $15 : 16 = 0$ resto 11

Dunque 3883 corrisponde a $F2B_{16}$.

Le conversioni binario/ottale e binario/esadecimale risultano semplici se si considera la Tabella DW.2.

Per convertire da binario a ottale si devono raggruppare le cifre in gruppi di tre a partire da destra e sostituire ogni gruppo con il corrispondente valore in base otto. Osserviamo la conversione di 1725 da binario 11010111101_2 a ottale:

1	1	0	1	0	1	1	1	1	0	1		binario
3			2			7			5			ottale

e la sua conversione in base sedici, in cui le cifre devono essere prese a gruppi di quattro in quattro:

1	1	0	1	0	1	1	1	1	1	0	1		binario
6				B				D					esadecimale

Con il procedimento inverso, sostituendo a ogni cifra il corrispondente valore binario, si ottiene anche la conversione da ottale o esadecimale in base due:

$3275_8 = 11.010.111.101_2 = 11010111101_2$
 $6BD_{16} = 110.1011.1101_2 = 11010111101_2$

Il sistema binario è quello che meglio si presta a essere utilizzato dagli elaboratori, in quanto è il più semplice possibile, avendo solo due simboli; d'altro canto, i numeri espressi in base due richiedono un numero di cifre più alto rispetto a quelli espressi in altre basi, come la base dieci, con una conseguente maggiore difficoltà per l'uomo di tenerli a mente e visualizzarne la dimensione. La facilità con cui si convertono tra loro i numeri in base binaria, esadecimale e ottale giustifica l'adozione di queste basi per rappresentare i numeri e gli indirizzi di memoria. Infine, notiamo che una sola cifra ottale rappresenta proprio il contenuto di un byte.

D.4 Complemento a 2

Per la rappresentazione dei numeri negativi, nei sistemi di elaborazione si utilizza spesso il *complemento a 1*, che consiste nel convertire le cifre binarie da 0 a 1 e viceversa. Vediamo un esempio, supponendo di avere a disposizione *parole* (*word*) di n bit, con n uguale a 16 bit o 2 byte, per memorizzare un numero intero. La *parola* è un raggruppamento di byte e può essere considerata l'insieme dei bit che un elaboratore può trattare in una sola operazione; le parole dei computer attuali sono più grandi di 16 bit: 32, 64, 128...

0	0	0	0	0	1	1	0	1	0	1	1	1	1	0	0	binario di 1724
1	1	1	1	1	0	0	1	0	1	0	0	0	0	1	1	complemento a 1

Il numero che otteniamo corrisponde alla differenza tra il massimo numero rappresentabile con il numero di bit a disposizione ($2^n - 1$, nell'esempio $2^{16} - 1 = 65535$) e il numero originario.

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	–	65535	–
0	0	0	0	1	1	0	1	0	1	1	1	1	0	0	=	1724	=	
1	1	1	1	1	0	0	1	0	1	0	0	0	1	1		63811		

Il complemento a 1, come abbiamo visto nel Capitolo 18 introducendo gli operatori binari del C, può anche essere calcolato esplicitamente dal programmatore con l'operatore bitwise `~`. Per ottenere il *complemento a 2* dobbiamo aggiungere 1:

1	1	1	1	1	0	0	1	0	1	0	0	0	1	1	+	complemento a 1	
															1	=	
1	1	1	1	1	0	0	1	0	1	0	0	1	0	0		complemento a 2	

La somma $1 + 1$ fa 0 e riporta 1, nella seconda colonna si ha ancora $1 + 1$: l'uno del primo addendo più lo zero che si riportava, che fa zero e riporta 1. Nella terza colonna si ha $0 + 1$: lo zero del primo addendo più l'uno che si riportava, che fa 1 e riporta zero. Da qui in avanti il risultato rimane uguale al primo addendo. La regola complessiva per ottenere il complemento a 2 è:

1. si convertono tutte le cifre binarie da 0 a 1 e viceversa (complemento a 1);
2. partendo da sinistra si convertono gli 0 in 1 e viceversa fino a che non si incontra il primo 0, che è l'ultima cifra che deve essere convertita.

Il metodo è equivalente al seguente altro:

1. partendo da destra (cifre meno significative) si lasciano inalterate tutte le cifre binarie fino a incontrare il primo 1;
2. si convertono tutti i bit successivi da 0 in 1 e viceversa.

0	0	0	0	0	1	1	0	1	0	1	1	1	1	0	0	binario di 1724
1	1	1	1	1	0	0	1	0	1	0	0	0	1	0	0	complemento a 2

Quella che si ottiene è la rappresentazione negativa del numero, nell'esempio – 1724. Supponiamo di voler trovare il risultato della sottrazione $2037 - 1724$; per farlo, addizioneremo a 2037 il complemento a 2 di 1724.

0	0	0	0	0	1	1	1	1	1	1	0	1	0	1	+	2037
1	1	1	1	1	0	0	1	0	1	0	0	0	1	0	=	-1724
0	0	0	0	0	0	1	0	0	1	1	1	0	0	1		313

L'ultimo riporto nella colonna più a sinistra viene ignorato. Come controprova della bontà del procedimento addizioniamo $1724 + -1724$: dovremmo ottenere zero.

0	0	0	0	0	1	1	0	1	0	1	1	1	1	0	0	+	1724
1	1	1	1	1	0	0	1	0	1	0	0	0	1	0	0	=	-1724
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		risultato uguale a zero

Perché il procedimento funziona? Per capirlo pensiamo in decimale, confidando nel fatto che il ragionamento sia lo stesso in ogni base. Siamo tutti d'accordo che:

$$\begin{aligned}
 2037 - 1724 &= \\
 &= 2037 - 1724 + 10000 - 10000 = \\
 &= 2037 + (10000 - 1724) - 10000 = \\
 &= 2037 + 8276 - 10000 = \\
 &= 10313 - 10000 = \\
 &= 313
 \end{aligned}$$

Ebbene, il risultato dell'espressione tra parentesi ($10000 - 1724$) è il complemento a 10 di 1724, dunque la sottrazione di $2037 - 1724$ può essere così effettuata:

1. si calcola il complemento a 10 del sottraendo ($10000 - 1724$);
2. si addiziona il minuendo 2037 al complemento a 10 del sottraendo 8276;
3. si sottrae al risultato precedente, 10313, la potenza del 10 immediatamente superiore al minuendo, 10000.

Il punto 3 può essere ottenuto anche in un altro modo:

3bis. si trascura la cifra più significativa del risultato precedente.

In entrambi i casi si ottiene 313. Il complemento a 10 in base decimale equivale al complemento a 2 in base binaria. Questo modo di procedere permette una rappresentazione chiara dei numeri negativi e consente di effettuare le sottrazioni eseguendo somme, oltre che i semplici procedimenti di scorrimento e modifica dei bit che abbiamo enunciato.

L'operazione di sottrazione sarebbe più complessa, in quanto dovrebbe considerare il “prendere a prestito” che conosciamo bene quando la eseguiamo manualmente.

Negli elaboratori, nella rappresentazione in complemento a 2 il bit all'estrema sinistra, quello più significativo, pur potendo essere trattato nelle operazioni alla stregua di tutti gli altri, indica il segno del numero: 0 per positivo, 1 per negativo. Dunque con una parola di 16 bit i *pesi* associati alle cifre binarie sono i seguenti:

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	bit
16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	posizione
-32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1	peso

Vediamo alcune rappresentazioni.

0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	32767	(16384 + 8192 + 4096 + ... + 1)
0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1	16387	(16384 + 2 + 1)
0 0 0 0 0 1 1 0 1 0 1 1 1 1 0 0	1724	(1024 + 512 + 128 + 32 + 16 + 8 + 4)
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	-1	(-32768 + 32767)
1 1 1 1 1 0 0 1 0 1 0 0 0 1 0 0	-724	(-32768 + 31044)
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1	-32767	(-32768 + 1)
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0	-32767	

Ecco perché, come avevamo detto nel Capitolo 6 introducendo gli `int`, con 2 byte sono rappresentabili valori compresi tra -215 (-32768) e +215 - 1 (+32767); in totale, quindi, 216 valori, considerando lo zero. In generale con n bit vengono rappresentati i numeri negativi da -1 a $-2^{n-1}-1$, i numeri positivi da 1 a $2^{n-1}-1$, oltre lo zero. Possiamo esprimere in linguaggio C come gli elaboratori effettuano la sottrazione: l'operazione:

```
c = a - b;
```

diventa:

```
c = a + (~b + 1);
```

D.5 Virgola mobile

I numeri frazionari possono venire rappresentati in *virgola fissa* (*fixed point*), stabilendo una posizione fissa in cui mettere la virgola – il punto per gli anglosassoni – nella parola per separare la parte intera da quella decimale. Nel caso di una parola di 32 bit si potrebbe pensare di utilizzare i 2 byte a sinistra per la parte intera e i 2 byte a destra per la parte decimale. Esiste tuttavia una scelta migliore in termini di occupazione di memoria e, quindi, di rango di numeri rappresentabile a parità di bit, quella della *virgola mobile* (*floating point*), cui abbiamo fatto riferimento nel Capitolo 6 introducendo il tipo `float`. Tale rappresentazione deriva dalla *rappresentazione scientifica* o standard, in cui un numero è il prodotto di

due parti: la seconda, detta *fattore di scala*, è una potenza di 10, l'altra, detta *parte frazionaria*, è un numero tale che moltiplicato per il fattore di scala restituisce il numero che vuole rappresentare. Dalla definizione discende che esistono più rappresentazioni dello stesso numero; per esempio sono equivalenti tra loro le rappresentazioni:

$$7,824 \cdot 10^3$$

$$78,24 \cdot 10^2$$

$$782,4 \cdot 10^1$$

$$78240 \cdot 10^{-1}$$

Viene però utilizzata una *rappresentazione normalizzata*, in cui la parte frazionaria è minore di 1 e la cifra più significativa è diversa da zero. La rappresentazione normalizzata del numero precedente è dunque

$$0,7824 \cdot 10^4$$

La rappresentazione in virgola mobile è la rappresentazione scientifica normalizzata con l'utilizzo del sistema binario, dunque il fattore di scala è una potenza del 2. La parte frazionaria è detta *mantissa*, l'esponente della potenza del 2 è detta *caratteristica* o anche *esponente*.

$$\pm \text{mantissa} \cdot 2^{\pm \text{esponente}}$$

La normalizzazione fa sì che non sia necessario rappresentare nel sistema di elaborazione la posizione della virgola, sempre prima della mantissa, e neppure la base del fattore di scala, sempre 2. Il modo in cui vengono memorizzate mantissa ed esponente varia da sistema a sistema.

Spesso, per evitare l'uso del segno nell'esponente, quest'ultimo viene rappresentato con un *eccesso*. Per esempio, se avessimo a disposizione 7 bit per l'esponente potremmo avere un eccesso a 64 facendo in modo che i numeri da 0 a 63 rappresentino gli esponenti negativi, il numero 64 rappresenti l'esponente zero e i numeri da 65 a 127 rappresentino gli esponenti positivi. Dunque prima di utilizzare la caratteristica così com'è memorizzata deve essere da essa sottratto l'eccesso, il numero 64.

segno	esponente	mantissa
0	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 ...	31

Nell'esempio abbiamo una parola di 32 bit, in cui il primo bit viene riservato per il segno della mantissa (attenzione: non dell'esponente, che invece è rappresentato con un eccesso a 64). La mantissa ci dà la *precisione* del numero, per cui è importante che per essa venga riservato maggior spazio possibile. D'altra parte l'esponente ci dà il limite del molto grande esponente positivo, e del molto piccolo esponente negativo cui possiamo arrivare.

D.6 Sistemi di codifica

Tutti i dati possono essere rappresentati o *codificati* con solo due simboli, con un computer: testi, suoni, immagini... Per i simboli che costituiscono i testi, i caratteri alfanumerici come le lettere dell'alfabeto, i numeri ma anche la punteggiatura e

**APPROFONDIMENTO****Il numero dei simboli**

256 simboli sembrano molti, ma non sono niente se si pensa alle lingue che non usano l'alfabeto, come il cinese, il giapponese o l'arabo. Pertanto è stato introdotto l'*Unicode*, un codice che opera su due byte rendendo disponibili $2^{16} = 65\,536$ simboli.

quant'altro, si stabilisce una corrispondenza che associa a ogni simbolo un numero rappresentato in base due. Il codice ASCII (*American Standard Code for Information Interchange*) sviluppato dall'ANSI, l'Istituto Nazionale Americano per gli Standard, utilizza 7 bit (8 nelle versioni estese), per cui vi si possono rappresentare $2^7 = 128$ simboli o $2^8 = 256$ simboli nelle versioni estese, in tal modo si possono eventualmente codificare, oltre alle lettere maiuscole e minuscole, alle cifre e ai segni di punteggiatura, anche una serie di caratteri aggiuntivi molto utili nei sistemi di elaborazione (si veda l'Appendice E).

Anche le immagini, i suoni, la musica e i film sono rappresentabili attraverso codici binari, con un procedimento noto come *digitalizzazione*, cioè trasformazione da analogico a digitale, dal continuo al discreto. Spesso il linguaggio C, proprio per le sue caratteristiche di manipolazione dei bit, viene utilizzato per realizzare sistemi per la riproduzione o la manipolazione di musica e film.

Verifica le conoscenzeSoluzioni sul sito web www.ateneonline.it/guidi5e 

1. Che cosa distingue i sistemi posizionali da quelli additivi? C'è anche qualcosa che li accomuna?
2. Dato un numero costituito da n cifre: $c_n c_{n-1} c_{n-2} \dots c_3 c_2 c_1$, espresso in base b , descrivere attraverso un'espressione matematica la sua conversione in base dieci.
3. Spiegare il modo più rapido per convertire mentalmente la base binaria nelle basi otto e sedici. Si ritiene che esistano altre basi aventi una relazione simile con la base binaria? Se sì, indicarne una e spiegare come avviene la conversione.
4. Scegliere un numero qualsiasi in base dieci e trasformarlo in base tre, sette, nove e venti.
5. Verificare l'esattezza di quanto fatto in risposta al punto precedente effettuando le conversioni inverse.
6. Prendere due numeri binari e sottrarre il primo dal secondo utilizzando il metodo del complemento a 2. Verificare l'esattezza dell'operazione convertendo i due numeri in base dieci.
7. Considerare due numeri decimali e sottrarre il primo dal secondo utilizzando il metodo del complemento a 10.
8. Spiegare la notazione scientifica e la sua relazione con la notazione a virgola mobile utilizzata nei sistemi di elaborazione.

9. Che cos'è il codice ASCII? E il codice Unicode?
10. Esiste un ordinamento tra i simboli del codice ASCII (Capitolo 6)?
11. È possibile utilizzare i caratteri del codice ASCII per rappresentare i numeri?

Applica le abilità

Soluzioni sul sito web www.ateneonline.it/guidi5e 

1. Convertire in base binaria, ottale ed esadecimale i seguenti numeri: 11, 51, 923, 3854, 30001, 525401
2. Convertire in base decimale i seguenti numeri:

1101001_2	11001001_2	101001110_2
57_8	326_8	2174_8
35_{16}	FC_{16}	$2ADE_{16}$
1304_6	384_9	$9A_{11}$
3. Convertire in base ottale ed esadecimale i seguenti numeri binari: 11, 1011, 111100, 11010001, 1001011110, 111101001101
4. Convertire in base binaria i seguenti numeri:

73_8	563_8	1627_8
$3B_{16}$	CDE_{16}	$500A_{16}$